CSP-203: Software Tools & Technologies Lab

Chapter – 4

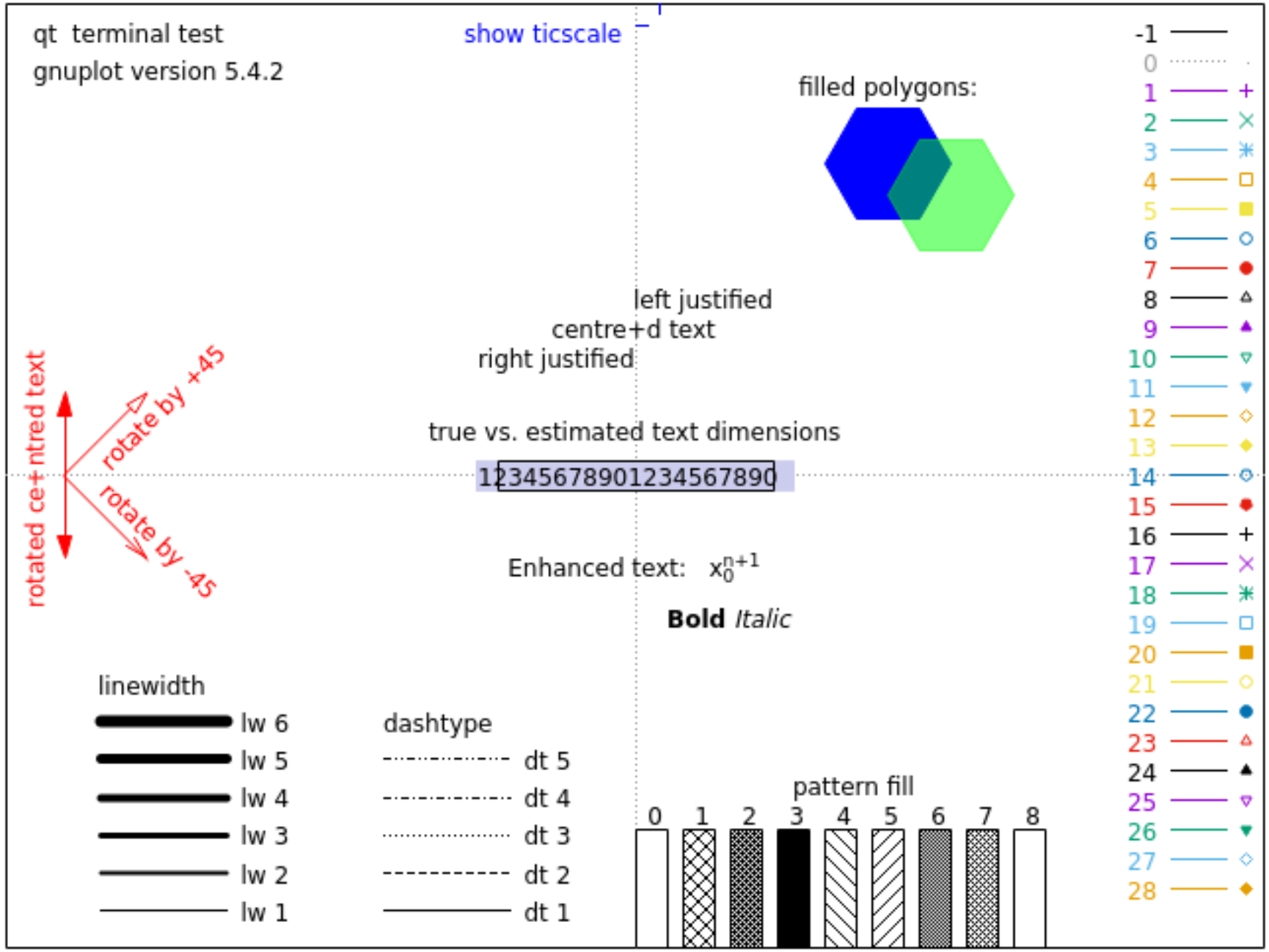
Plotting Tools: GNU Plot (Lecture – 2)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Title, labels, ticks
- ❑ Defining range
- ❑ Changing line colour and style
- ❑ Datafile handling
- ❑ Saving the figure
- ❑ Changing figure size
- ❑ Creating script to execute gnuplot

- **Line Styles available:**
  - lt 1: Solid, lt 2: Dashed, lt 3: Dotted ……
  - Example : plot sin(x) lt 2 title "Dashed Line"
- **Adjusting Line Colors:**
  - Example: plot cos(x) lc rgb "red" title "Red Line“ (Using color name)
  - Example: plot tan(x) lc rgb "#00FF00" title "Green Line" (Using hexadecimal code)
- **Adjusting Line Thickness:**
  - lw1: Default, lw 2: Medium, lw 3: Thick
- **Combining Style, Color, Thickness:**
  - plot cos(x) lt 2 lc rgb "blue" lw 2 title "Customized Line"


qt terminal test
gnuplot version 5.4.2
show ticscale
filled polygons:
left justified
centre+d text
right justified
rotated ce+ntred text
rotate by +45
rotate by -45
true vs. estimated text dimensions
12345678901234567890
Enhanced text: x0n+1
Bold Italic
linewidth
lw 6
lw 5
lw 4
lw 3
lw 2
lw 1
dashtype
dt 5
dt 4
dt 3
dt 2
dt 1
pattern fill
0 1 2 3 4 5 6 7 8
-1
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28

- **Plotting with specific point type:**
  - plot sin(x) with points pt 7 => This plots the sine function using filled circles (pt 7) at each datapoint.
- **Combining line type with point type:**
  - plot sin(x) with linespoints pt 5 => This plots the sine function with lines connecting points, using filled squares (pt 5) at each datapoint.
- **Point size:**
  - plot sin(x) with points pt 7 ps 1.5 => This plots the sine function using point type 7 (Filled Circle) with a size 1.5 times the default.

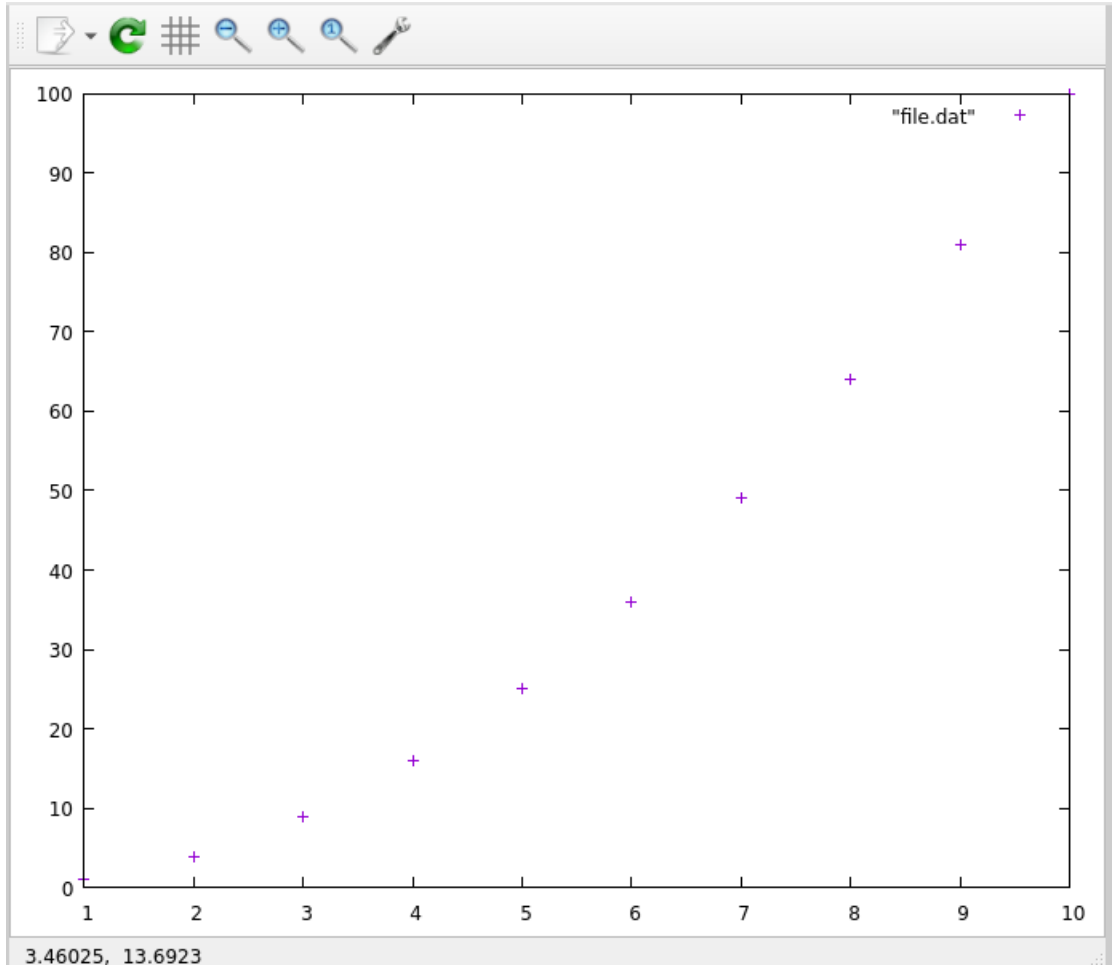
- **ASCII-Datafiles in multi-column format:**

file.dat

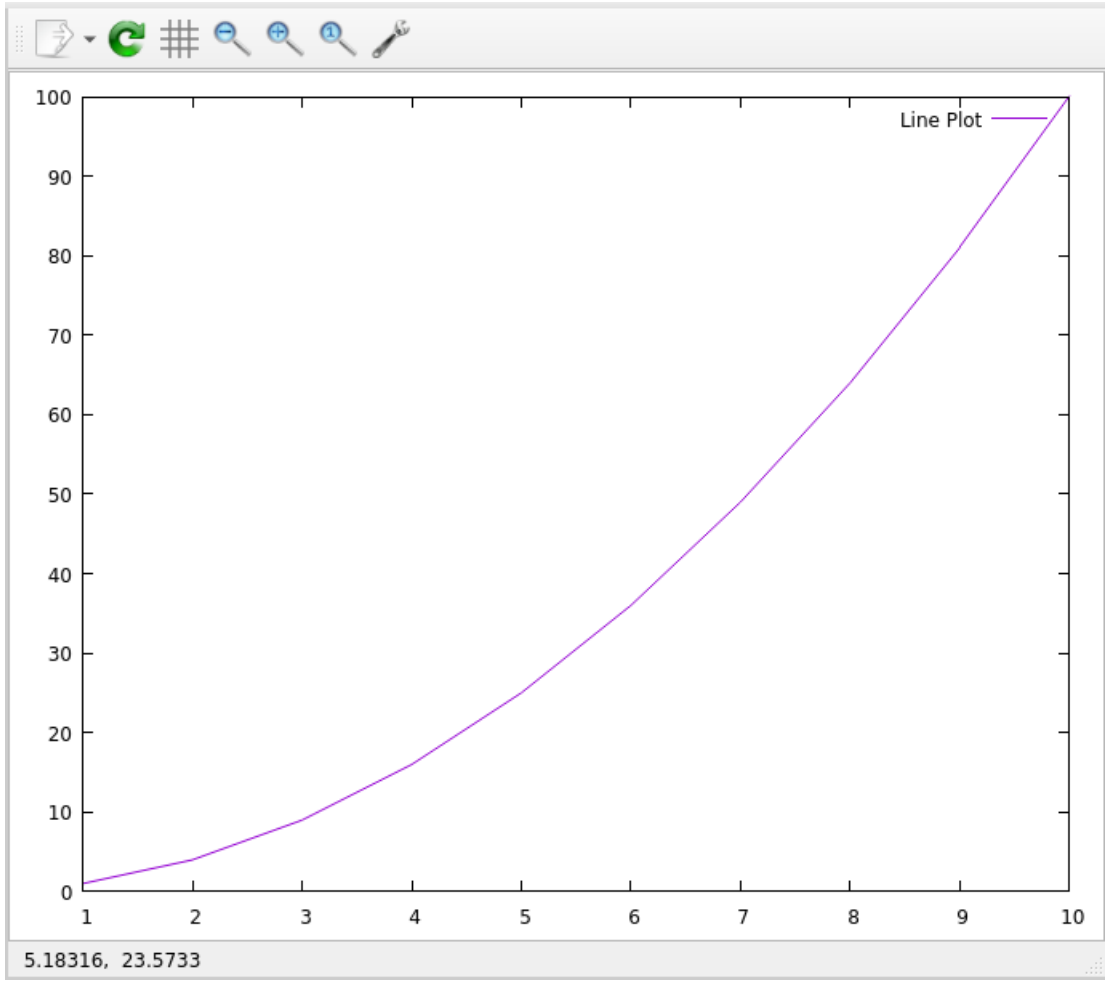
```
# Comments can be placed here
x0  y0  z0   ...
x1  y1  z1   ...
.    .   .   ...
xN  yN  zN   …
```

- To plot a multi-column datafile using the 1st column for the abscissa (x-value) and the 2nd column as the ordinate (y-value), use **:** plot “file.dat” using 1:2
- Add a second plot using 1st (=x) and 3rd (=z) columns: replot “file.dat” using 1:3
- If the “using” keyword is not specified, 1st and 2nd columns are assumed: plot “file.dat”

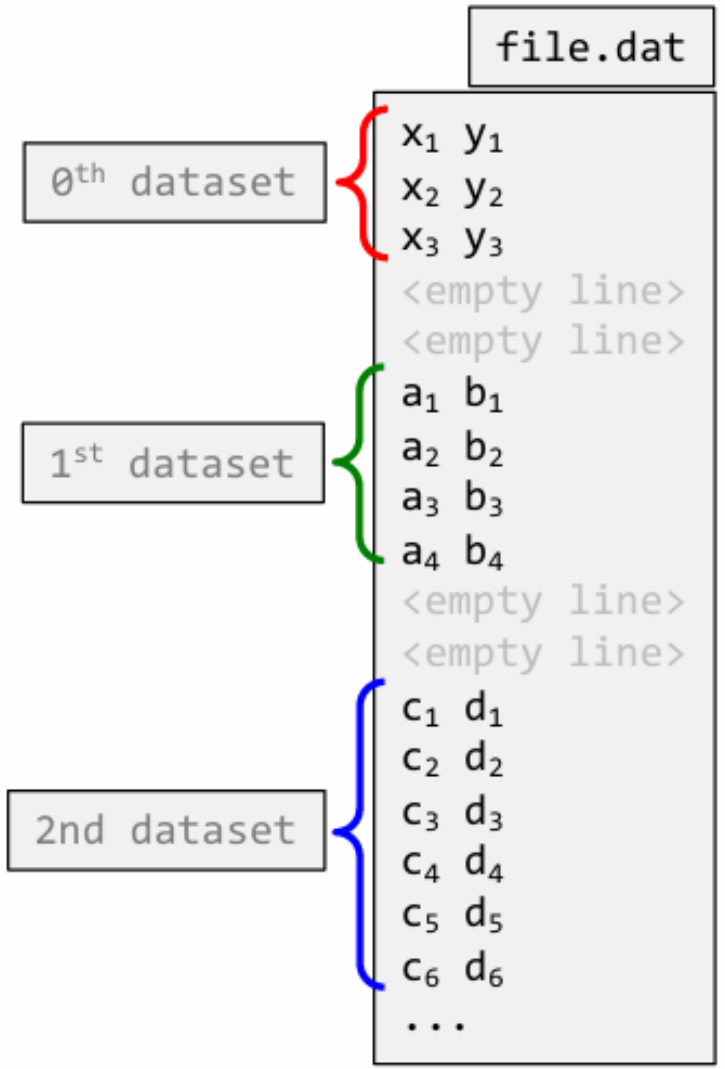
- **When plotting datafiles, gnuplot uses symbols:** plot “file.dat”



- **When plotting datafiles, gnuplot uses symbols and join symbol with lines:** plot "file.dat" with lines



- A single datafile may also include more than one data set, which must be separated by a pair of empty lines
- In this case you can tell gnuplot which dataset should be read using the ‘index’ keyword.
- Example: plot “file.dat” using 1:2 index 0 will plot the 3 points (x1, y1), (x2, y2), (x3,y3).
- Example: plot “file.dat” using 1:2 index 2 will plot the 6 points (c1, d1), (c2, d2) ... (c6,d6).



- Data file separator: By default, gnu-plot treats tabs and white spaces as field value separator.
- If we want to specify the field value separator manually, we have to use this command: set datafile separator “<separator_name>".

- In GNU-plot, you can control the spacing of tick marks on graph's axis using xtics and ytics command.
- Syntax for setting x-axis tick intervals:
  - set xtics <start>, <increment>, <end>
- Syntax for setting y-axis tick intervals:
  - Set ytics <start>, <increment>, <end>
- Automatic tick Generation: set xtics auto
- Custom tick labels: To assign specific labels to tick positions
  - set xtics ("Label1" 1, "Label2" 2, "Label3" 3)
- Rotating tick labels: set xtics rotate by 45
- Offsetting tick labels: set xtics offset 0, -1

- **Saving figure in GNU-plot:**
  - To save a figure in gnu-plot, you need to specify the terminal type (Output format) before your plot.
  - General Procedure:
    - set terminal pngcairo size 800,600 enhanced font 'Arial,10’

      set output 'output.png’

      plot 'data.dat' using 1:2 with lines title 'My Plot’

      unset output
  - Things to remember:
    - Always use output command before the plot command.
    - Always unset output after plotting to flush the file and avoid corruption.

❑ Common Terminal Types:

Format	Terminal Command	File Extension
PNG	set terminal pngcairo	.png
PDF	set terminal pdfcairo	.pdf
SVG	set terminal svg	.svg
PostScript	set terminal postscript eps	.eps

- **Setting Figure Size:**
  - Set the output canvas size
    - Command: set terminal <type> size <width>,<height>
  - Scale the plot within the canvas:
    - Full canvas: set size 1,1
    - Half canvas: set size 0.5, 0.5
    - Custom Canvas: set size 0.8, 0.6

❑ Creating script to execute gnu-plot:

- ➢ Create a script file: filename.gp
- ➢ Write the commands in the script file:

```
# Set the terminal type and output file
set terminal pngcairo size 800,600
set output 'line_graph.png’
# Set plot title and axis labels
set title 'Line Graph Example’
set xlabel 'X-Axis’
set ylabel 'Y-Axis’
# Plot the data
plot 'data.dat' using 1:2 with lines title 'Data Line’
```

- ➢ Execute the script: gnuplot line_plot.gp

CSP-203: Software Tools & Technologies Lab

Chapter – 4

Plotting Tools: Matplotlib Plot (Lecture – 2)

Dr. Anand M. Baswade
Indian Institute of Technology, Bhilai
anand@iitbhilai.ac.in

Contents

- ❑ Introduction
- ❑ Installing Matplotlib and About Section
- ❑ Line Plot
- ❑ Vertical Bar Plot
- ❑ Horizontal Bar Plot
- ❑ Bar Chart
- ❑ Histogram
- ❑ Scatter Plot
- ❑ Numpy Introduction
- ❑ Numpy Functions
- ❑ Pie Chart
- ❑ Xtics, Ytics
- ❑ Different Markers
- ❑ Color Codes
- ❑ Line Style and Custom Line Style

What is Matplotlib?

- Matplotlib is one of the most powerful tools for data visualization in python programming language.
- Is a 2D plotting library.
- Compatible with Linux, macOS and other operating systems.
- Matplotlib was first released in 2003.
- It was created by John D. Hunter as a way to generate plots in Python similar to those produced by MATLAB.

How to install Matplotlib through terminal?

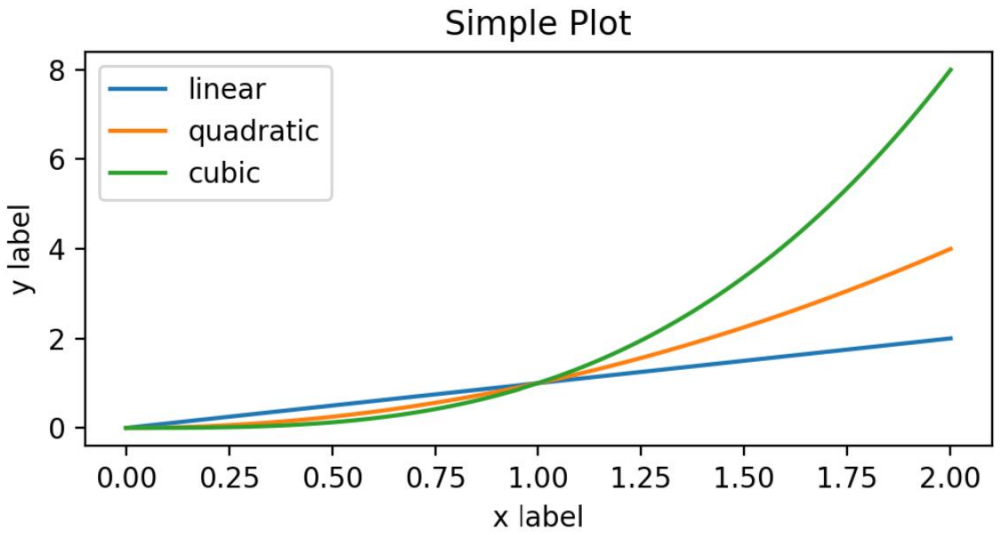
- Easiest way to install matplotlib in python is using pip.
- Type the command in Linux shell or Command Prompt of your operating system:
  - pip install matplotlib
  - Note: You may need to run the above cmd as an administrator.
  - Note: If your installation fails for some error or dependencies, then first execute the following command before again installing matplotlib:
    - sudo apt-get install update
    - sudo apt-get install upgrade

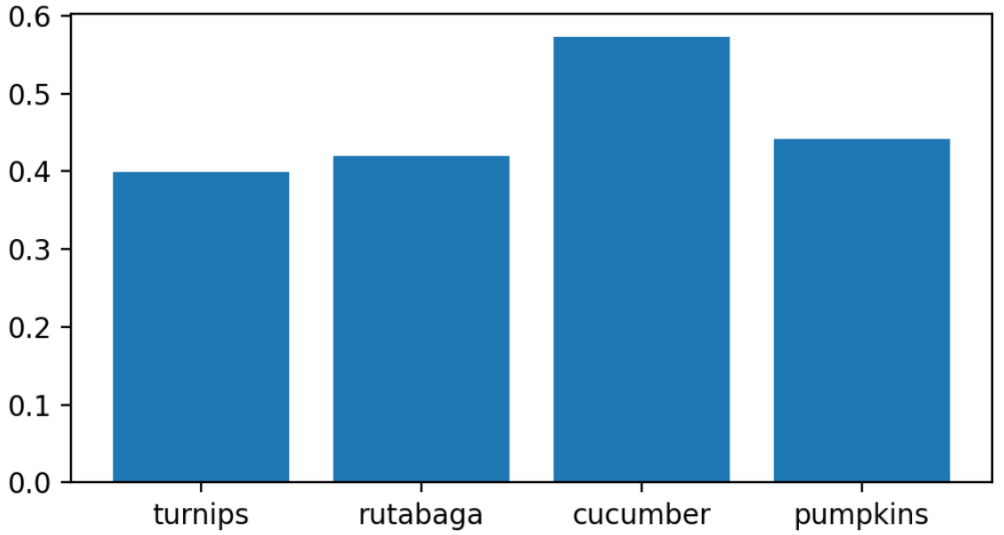
About Matplotlib?

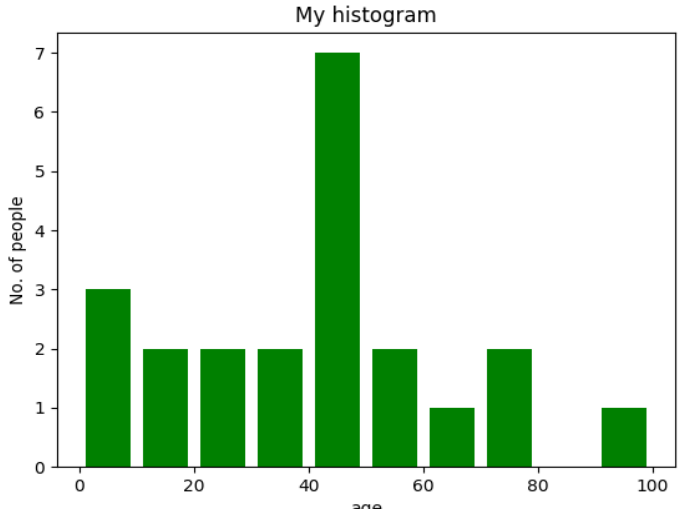
- In order to get matplotlib in your script:
  - Write the command in your python script before using matplotlib tools:
    - import matplotlib.pyplot as plt
  - Matplotlib.pyplot is a collection of command style functions that make matplotlib work like MATLAB's plotting functions.
    - Creates a figure and a plotting area in the figure
    - Plots some lines in the plotting area
    - Decorates the plot with colors, labels and styles
  - Note: Various states are preserved across function calls.
  - Plotting with matplotlib, the two main code lines should be considered:
    - Type of graph
    - Show the graph

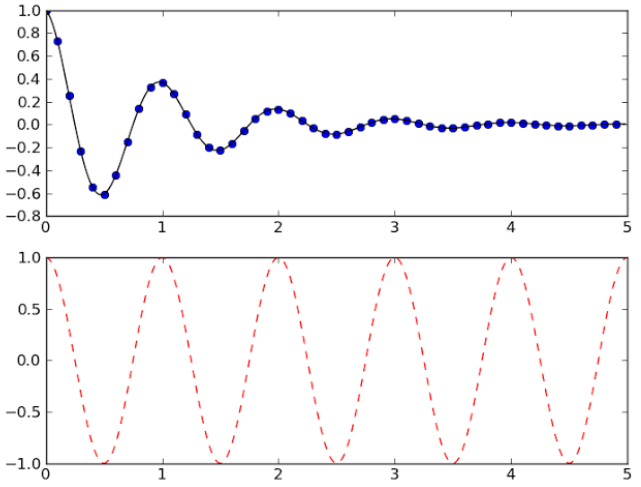
Examples of Matplotlib?

- It allows to make easy histograms, barcharts, scatterplots, line plots.



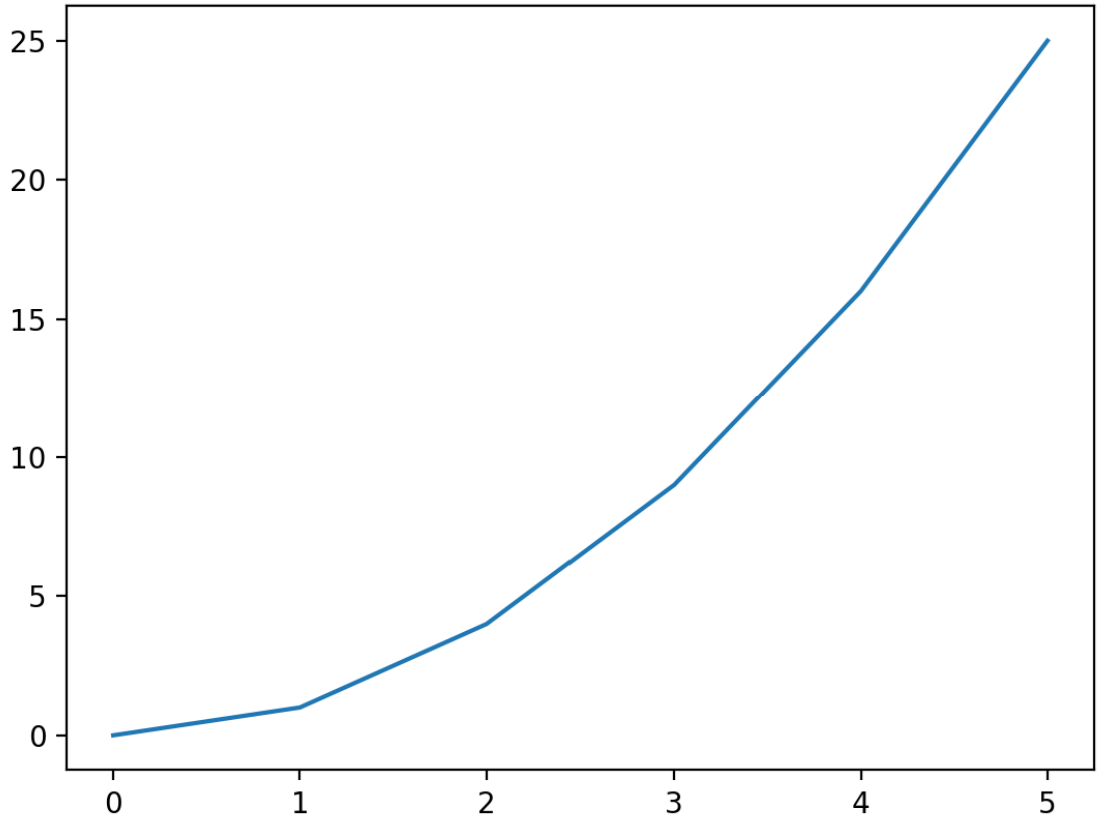






Plotting Line Plot using List:

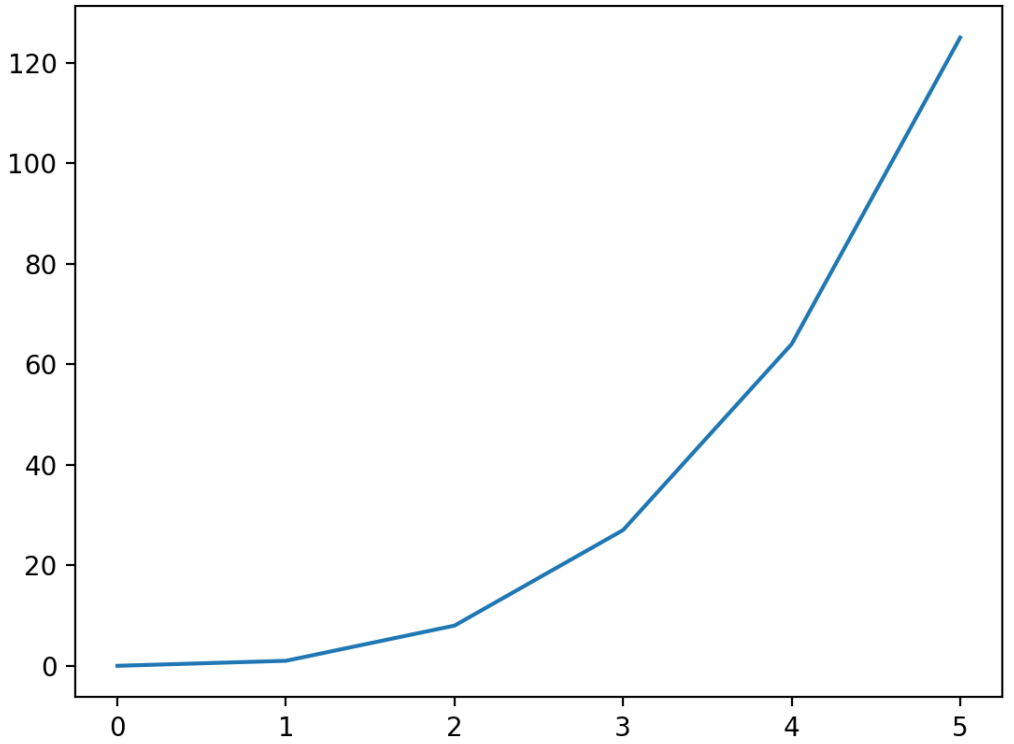
```
import matplotlib.pyplot as plt
#creating lists for x-axis and y-axis
x_val = [0, 1, 2, 3, 4]
y_val = [0, 1, 4, 9, 16]
#the default style for plotting line graph
plt.plot(x_val, y_val)
#display the graph
plt.show()
```



Plotting Line Graph using List:

- Note: If only single list or array is provided to plot() command
  - Matplotlib assumes it as y_val.
  - Automatically generates x_val.
  - Since python range starts with 0, the default x vector has same length as y but starts with 0.
  - Example:
    - import matplotlib.pyplot as plt

      plt.plot([0,1,8,64,125])
    - plt.show()

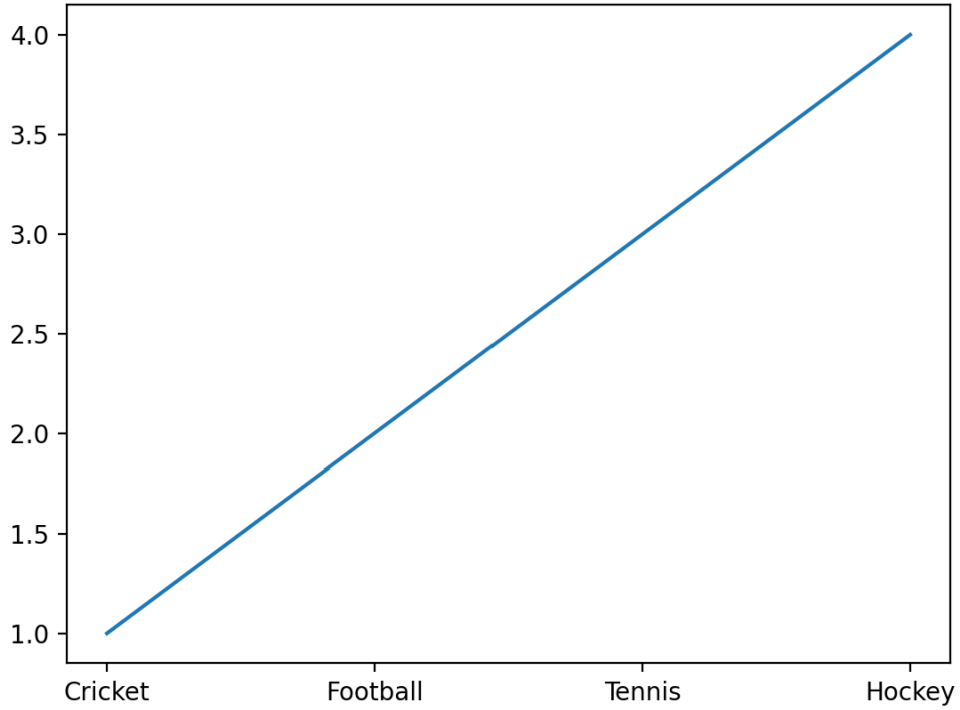


Plotting Line Plot using Dictionary:

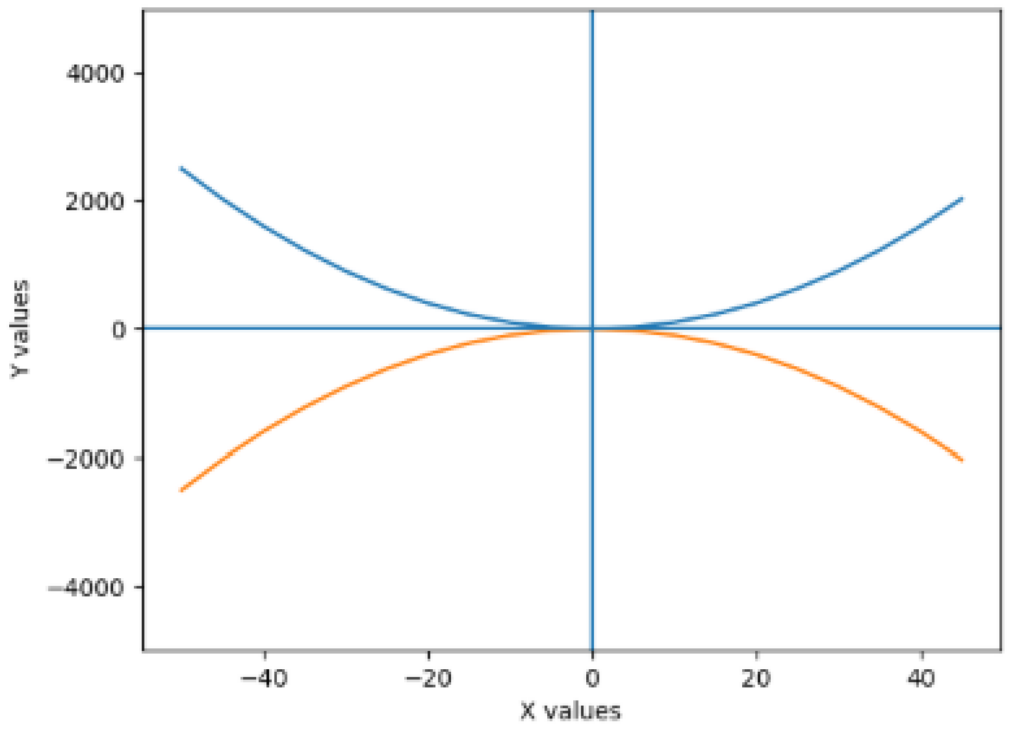
- import matplotlib.pyplot as plt
- dictionary =

  {"Cricket":1,"Football":2,"Tennis":3,"Hockey":

  4}
- plt.plot(dictionary.keys(),dictionary.values())
- plt.show()

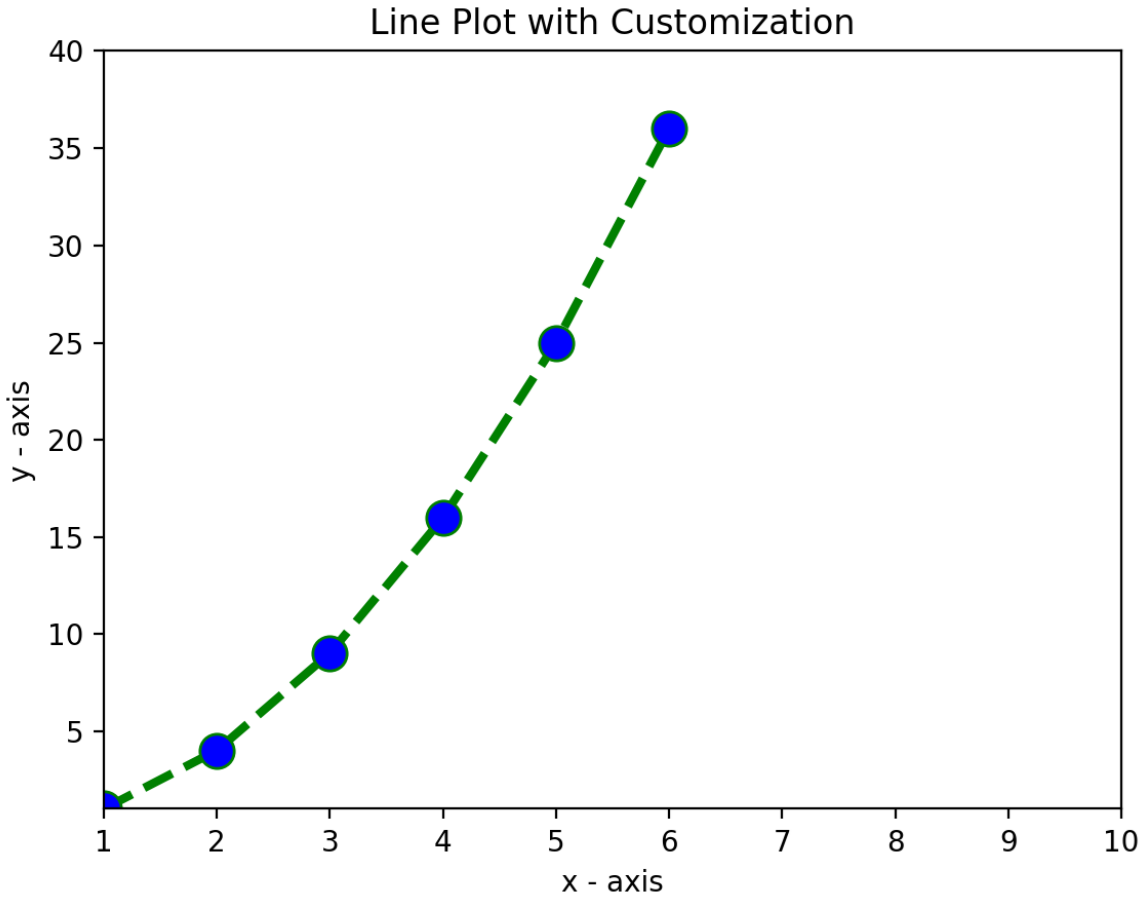


- text() : Adds text in an arbitrary location.
- xlabel() : Adds text to the x-axis.
- ylabel() : Adds text to the y-axis.
- title() : Adds title to the plot.
- clear() : Removes all plots from the axes.
- savefig() : Saves your figure to a file.
- legend() : Shows a legend on the plot
- All methods are available on pyplot.



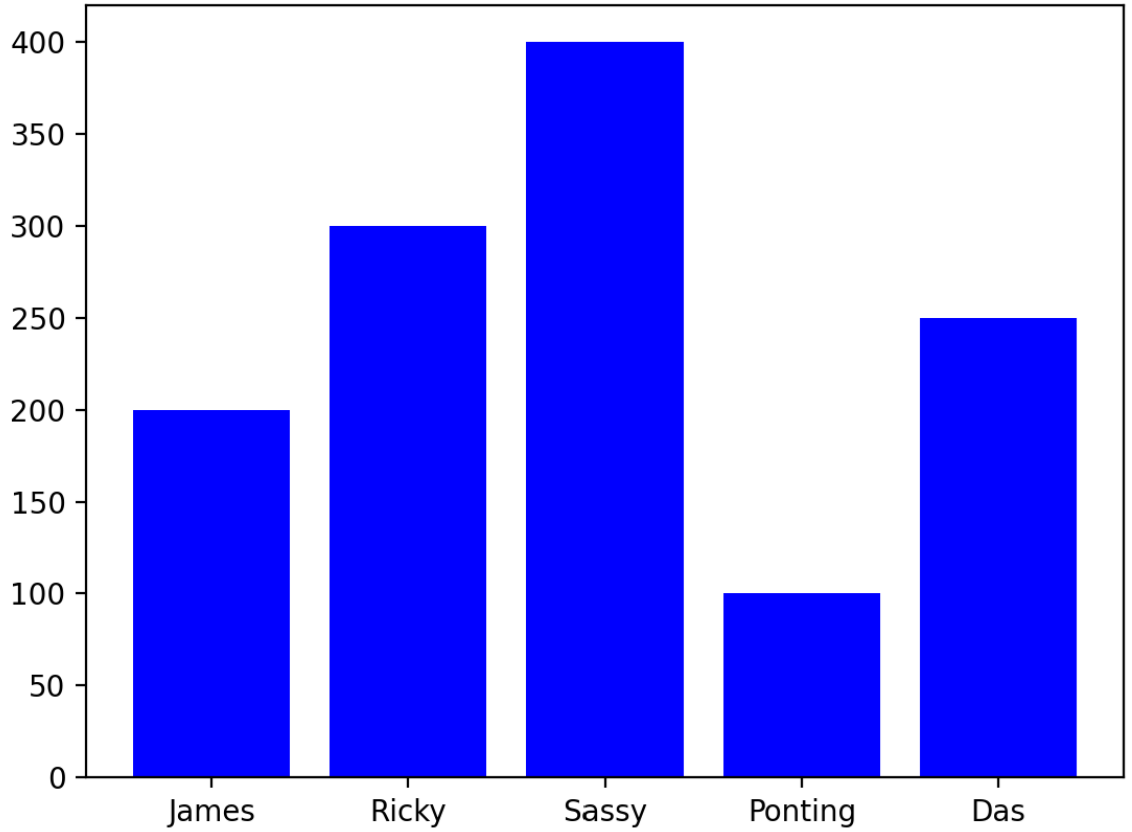
Customization of a line graph:

```
import matplotlib.pyplot as plt
# x axis values
x = [1,2,3,4,5,6]
# corresponding y axis values
y = [1,4,9,16,25,36]
# plotting the points
plt.plot(x, y, color='green', linestyle='dashed',
linewidth = 3, marker='o',
markerfacecolor='blue', markersize=12)
# setting x and y axis range
plt.ylim(1,40)
plt.xlim(1,10)
# naming the x axis
plt.xlabel('x - axis')
# naming the y axis
plt.ylabel('y - axis')
# giving a title to my graph
plt.title('Line Plot with Customization')
plt.show()
```



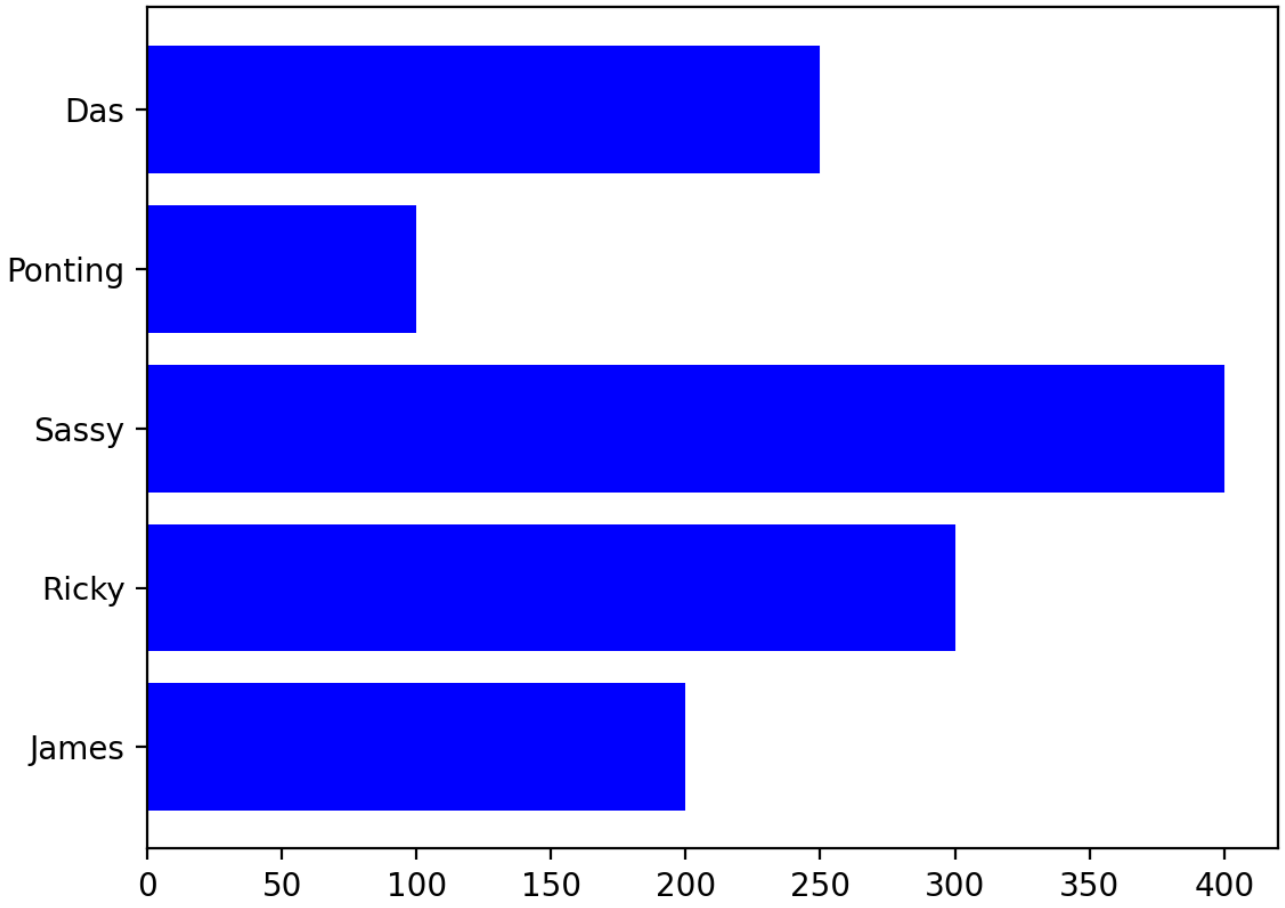
Vertical Bar Graphs:

- import matplotlib.pyplot as plt
- #Create data for plotting
- income = [200,300,400,100,250]
- names = ["James", "Ricky", "Sassy", "Ponting", "Das"]
- plt.bar(names, income, color="blue")
- plt.show()



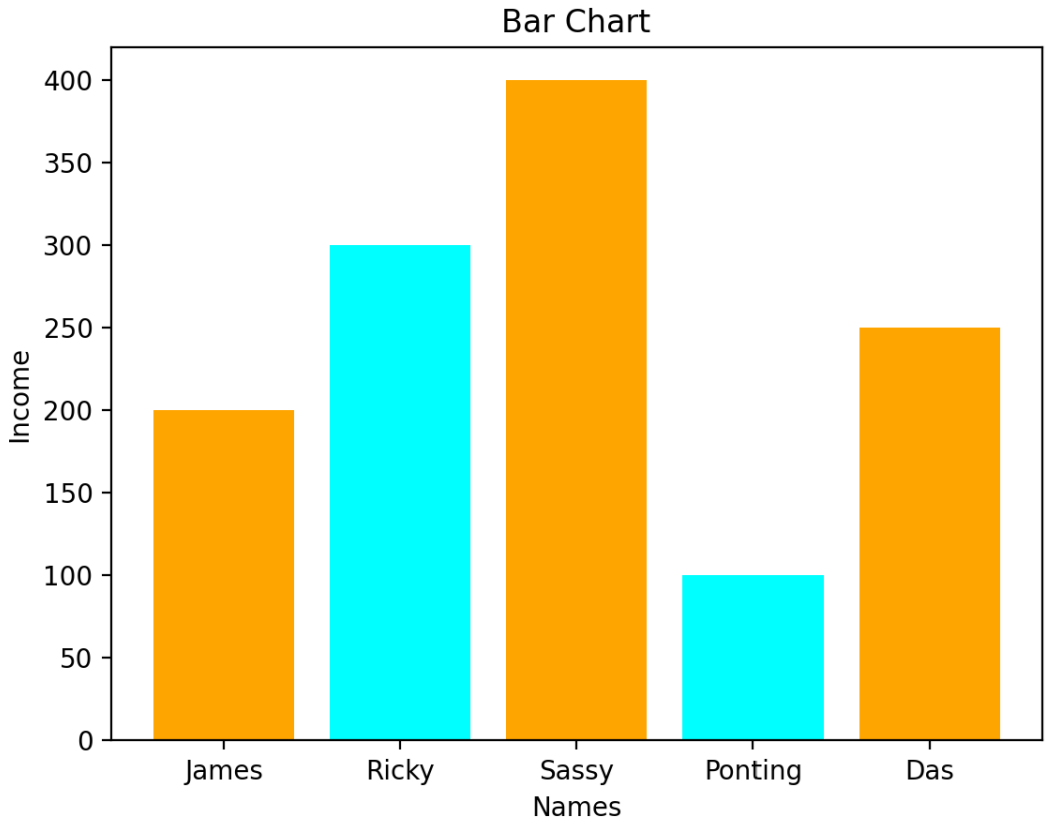
Horizontal Bar Graphs:

- import matplotlib.pyplot as plt
- #Create data for plotting
- income = [200,300,400,100,250]
- names = ["James", "Ricky", "Sassy", "Ponting", "Das"]
- plt.barh(names, income, color="blue")
- plt.show()



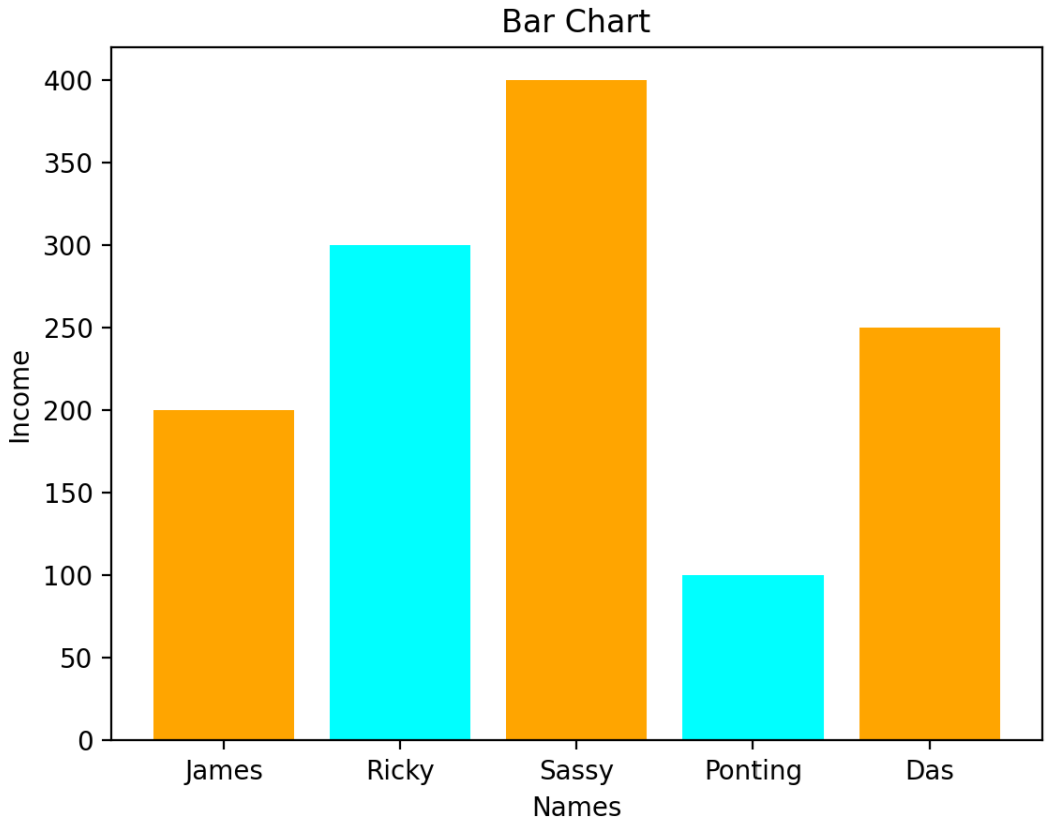
Bar Chart:

- import matplotlib.pyplot as plt
- #Create data for plotting
- income = [200,300,400,100,250]
- names = ["James", "Ricky", "Sassy", "Ponting", "Das"]
- c1 = ["orange","cyan"]
- c2 = ["b","r"]
- plt.bar(names, income, color=c1,width=0.8)
- plt.xlabel("Names")
- plt.ylabel("Income")
- plt.title("Bar Chart")
- plt.show()



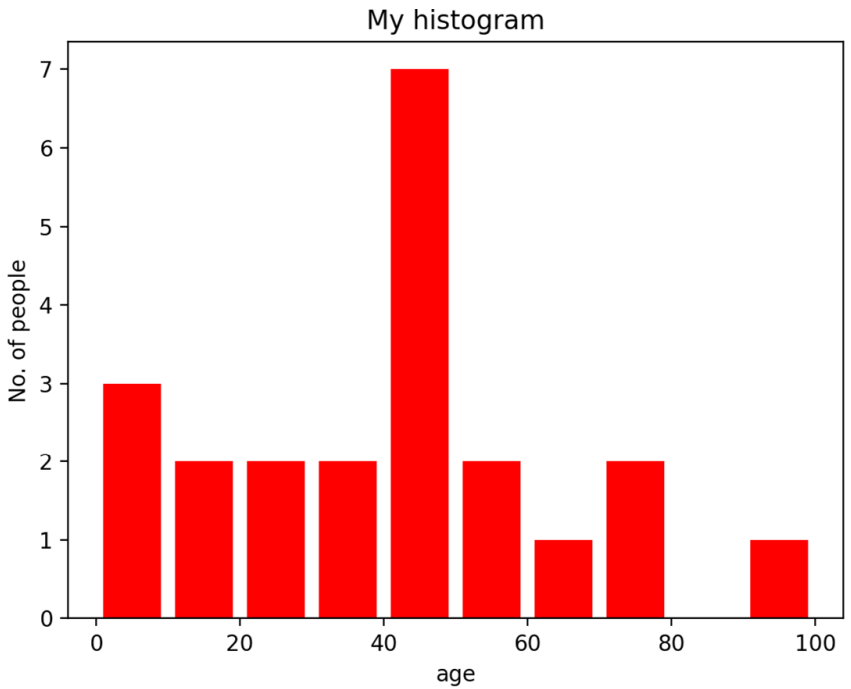
Explanation of Bar Chart:

- c1 = ["orange", "cyan"], c2 = ["b", "r"]
  - These are color lists.
  - c1 and c2 are not both used in the plot; only c1 is passed to the color parameter.
  - c1 contains only 2 colors, while there are 5 bars — this leads to color cycling: the colors will repeat from the beginning if not enough are provided.



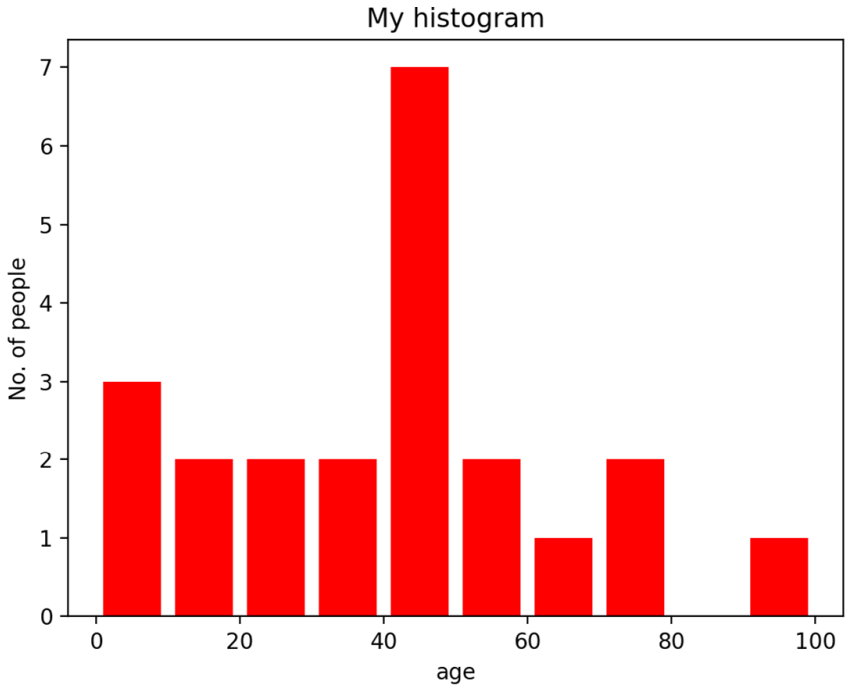
Histogram:

```
import matplotlib.pyplot as plt
# frequencies
ages=[2,5,70,40,30,45,50,45,43,40,44,60,7,13,57,18,90,77,32,21,20,40]
# setting the ranges and no. of intervals
range = (0, 100)
bins = 10
# plotting a histogram
plt.hist(ages, bins, range, color='red',histtype='bar',rwidth=0.8)
# x-axis label
plt.xlabel('age')
# frequency label
plt.ylabel('No. of people')
# plot title
plt.title('My histogram')
# function to show the plot
plt.show()
```



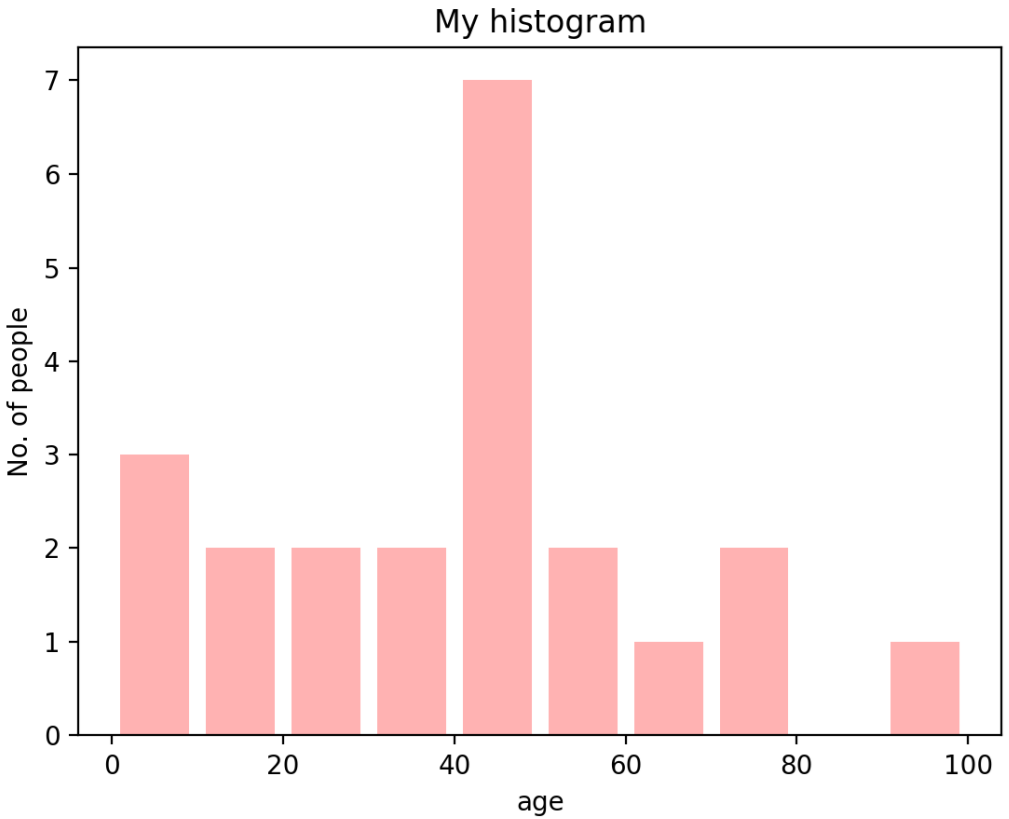
Histogram Explanation:

- ages = [...]: List of age values representing a sample dataset.
- Set Range and Bins
  - range = (0, 100): Histogram will cover ages 0 to 100.
  - bins = 10: Divides the range into 10 intervals of 10 years each.
- Plot Histogram
  - plt.hist(...): Plots the histogram with:
    - ages: Input data.
    - bins and range: Defines age groups.
    - color='red': Sets bar color.
    - histtype='bar': Bar-style display.
    - rwidth=0.8: Bars occupy 80% width (adds spacing).



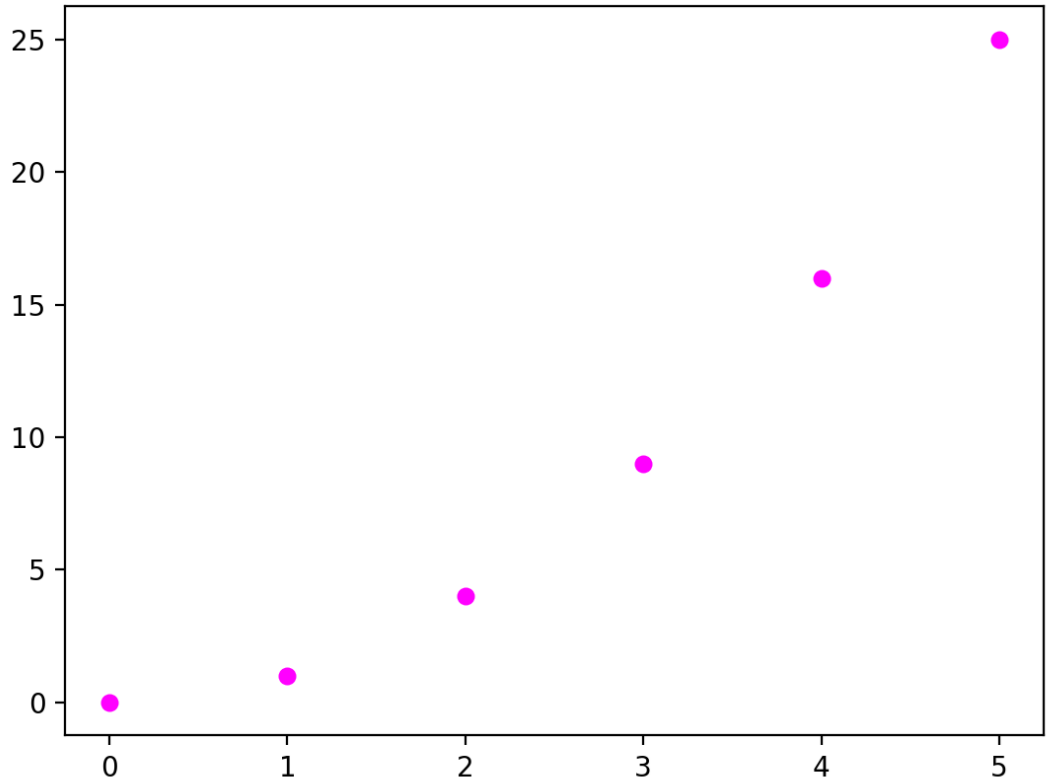
Histogram Transparency:

- Bins: It is an argument specific to a histogram and allows the user to customize how many bins they want
- Alpha: is an argument that displays the level of transparency of the datapoints.
- plt.hist(ages, bins, range, color='red',histtype='bar',rwidth=0.8, alpha=0.3)



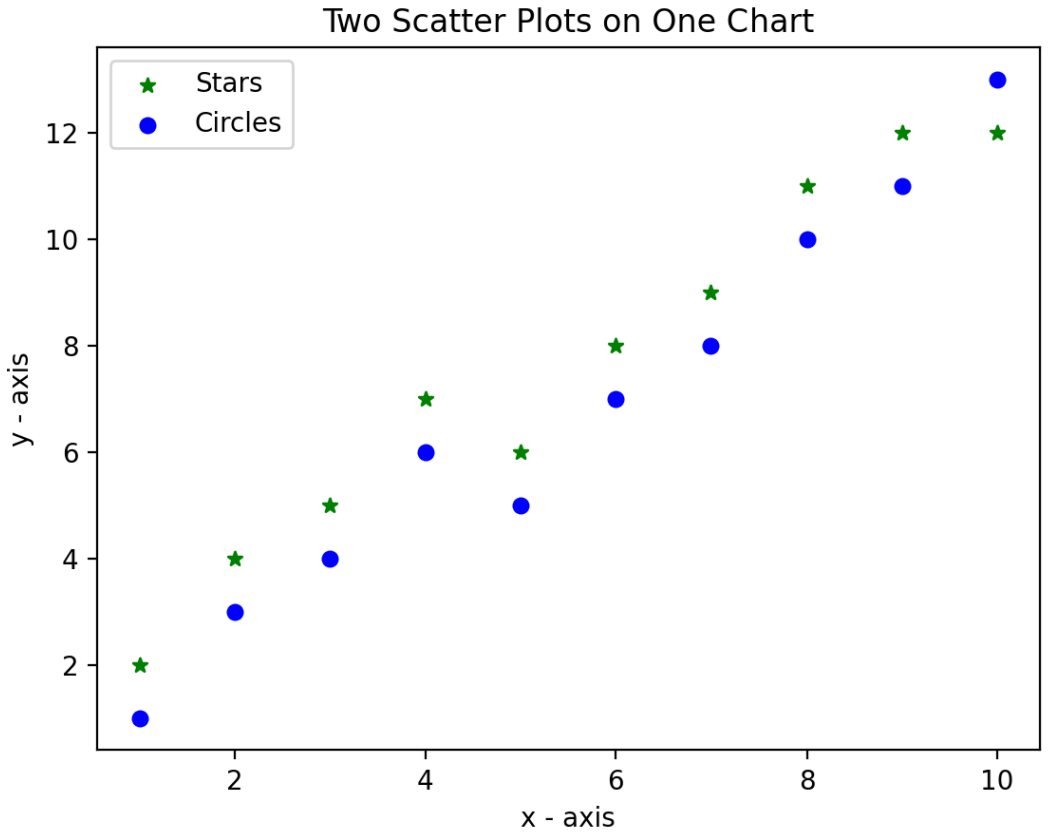
Scatter Plot:

- import matplotlib.pyplot as plt
- #create data for plotting
- x_values = [0,1,2,3,4,5]
- y_values = [0,1,4,9,16,25]
- plt.scatter(x_values, y_values, s=30, color=“magenta")
- plt.show()
- #Here size of each point is set as “s=30”



Two Scatter Plot:

- # First scatter plot
- plt.scatter(x1, y1, label="Stars", color="green", marker="*", s=30)
- # Second scatter plot
- plt.scatter(x2, y2, label="Circles", color="blue", marker="o", s=30)
- marker : It visually distinguishes data points by specifying what symbol to draw at each (x, y) coordinate.

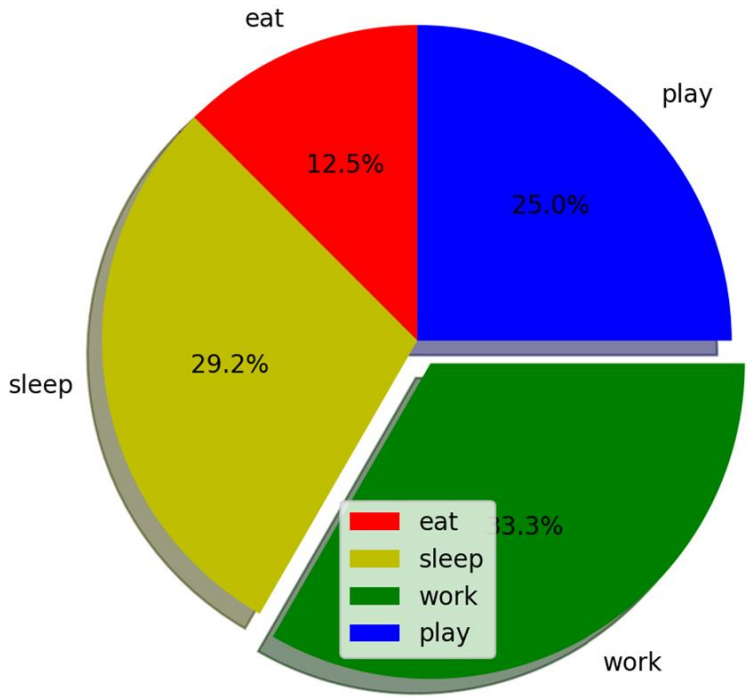


Plotting pie chart:

- import matplotlib.pyplot as plt
- activities = ['eat', 'sleep', 'work', 'play']
- slices = [3, 7, 8, 6]
- colors = ['r', 'y', 'g', 'b']
- plt.figure(figsize=(6, 6)) # Optional: Set figure size
- plt.pie(slices, labels=activities, colors=colors,

  startangle=90, shadow=True, explode=(0, 0, 0.1, 0), radius=1.2,

  autopct='%1.1f%%')
- plt.legend()
- plt.title("Daily Activities Distribution")
- plt.show()



Explanation of Pie Chart:

- Define Labels and Values
  - activities = ['eat', 'sleep', 'work', 'play']: Labels for each slice.
  - slices = [3, 7, 8, 6]: Proportions for each activity.
- Assign Colors
  - colors = ['r', 'y', 'g', 'b']: Colors for the slices (red, yellow, green, blue).
- Plot Pie Chartplt.
  - pie(...): Creates the pie chart with:
    - labels=activities: Sets slice labels.
    - colors=colors: Sets corresponding slice colors.
    - startangle=90: Rotates the chart start by 90°.
    - shadow=True: Adds a shadow for visual effect.
    - explode=(0, 0, 0.1, 0): "Pops out" the third slice (work).
    - radius=1.2: Enlarges the pie chart.
    - autopct='%1.1f%%': Displays values as percentages (1 decimal place).
    - Add Legend and Show Plotplt.legend(): Adds a legend.plt.show(): Displays the final chart.

What is numpy:

- NumPy stands for Numerical Python.
- It is a fundamental Python library for scientific computing.
- Provides powerful tools for working with:
  - Large, multi-dimensional arrays and matrices.
  - Mathematical functions to operate on these arrays efficiently.
- Key features:
  - Efficient storage and manipulation of numerical data.
  - Supports fast operations on arrays, faster than Python's built-in lists.
  - Offers many built-in mathematical functions (e.g., trigonometry, linear algebra).
  - Foundation for many other scientific libraries (like SciPy, Pandas, Matplotlib).
- Why use NumPy?
  - Simplifies handling of large datasets.
  - Speeds up calculations using optimized C code under the hood.Enables vectorized operations (operate on entire arrays without explicit loops).

Array Creation Functions Available in numpy:

Function	Description
np.array()	Create an array from a list or tuple.
np.zeros(shape)	Create an array filled with zeros.
np.ones(shape)	Create an array filled with ones.
np.eye(n)	Create an identity matrix of size n×n.
np.arange(start, stop, step)	Create array with evenly spaced values.
np.linspace(start, stop, num)	Create array with specified number of values.

Mathematical Functions Available in numpy:

Function	Description
np.sin(x), np.cos(x), np.tan(x)	Trigonometric functions
np.sinh(x), np.cosh(x), np.tanh(x)	Hyperbolic functions
np.exp(x)	Exponential function exe^xex
np.log(x)	Natural logarithm
np.log10(x)	Base-10 logarithm
np.sqrt(x)	Square root

Statistical Functions Available in numpy:

Function	Description
np.mean(array)	Calculate mean
np.median(array)	Calculate median
np.std(array)	Calculate standard deviation
np.var(array)	Calculate variance
np.min(array)	Minimum value in array
np.max(array)	Maximum value in array

Array Manipulation Functions Available in numpy:

Function	Description
np.reshape(array, new_shape)	Change shape of array
np.flatten()	Flatten multi-dimensional array to 1D
np.concatenate((arr1, arr2), axis=0)	Join arrays along a specific axis
np.transpose(array)	Transpose of an array (swap axes)

Linear Algebra Functions Available in numpy:

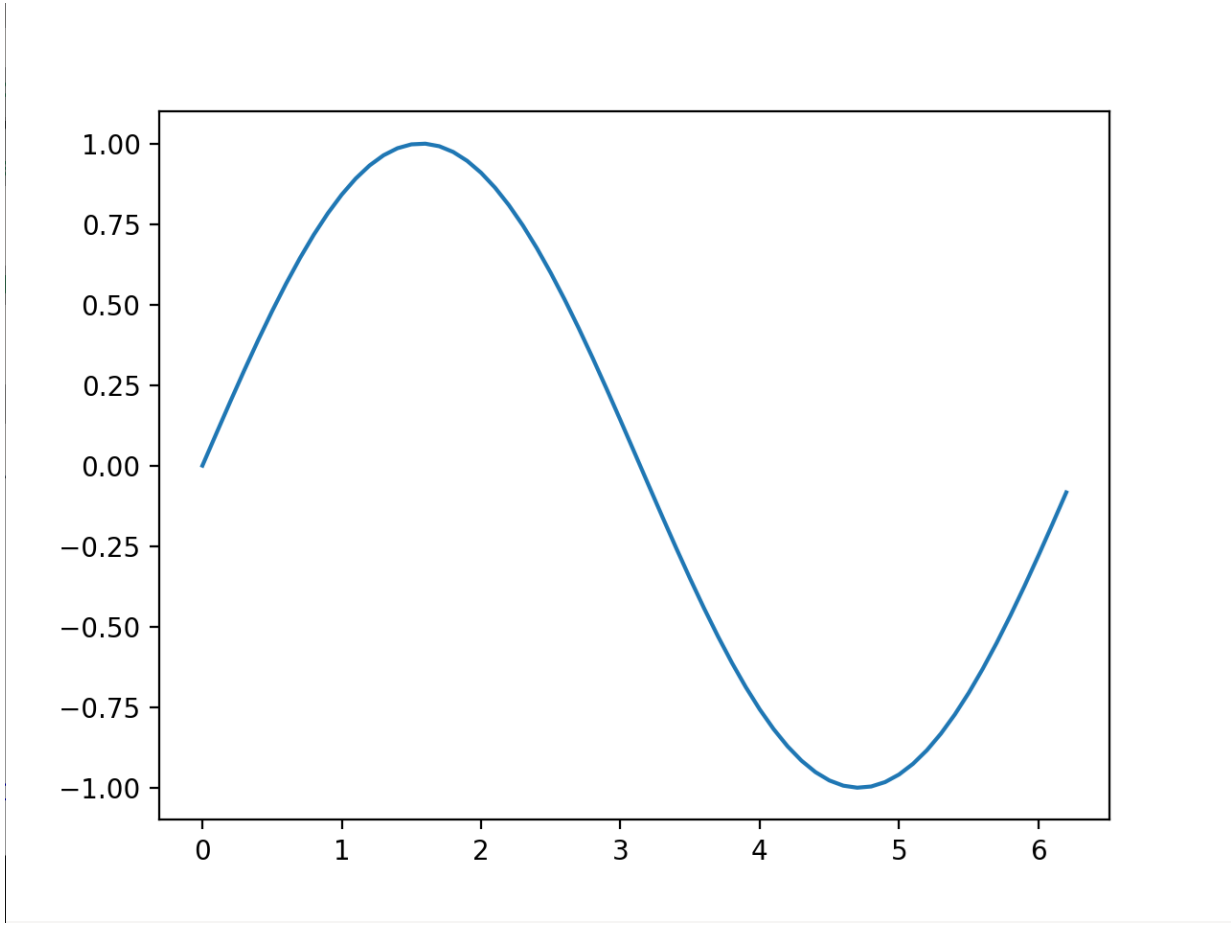
Function	Description
np.dot(a, b)	Dot product
np.matmul(a, b)	Matrix multiplication
np.linalg.inv(matrix)	Inverse of a matrix
np.linalg.det(matrix)	Determinant of a matrix
np.linalg.eig(matrix)	Eigenvalues and eigenvectors

Logical/Conditional Operation Functions Available in numpy:

Function	Description
np.where(condition, x, y)	Return elements based on condition
np.any(array)	True if **any** element is True
np.all(array)	True if **all** elements are True

Plotting curves of equations:

- # importing the required modules
- import matplotlib.pyplot as plt
- import numpy as np
- # setting the x - coordinates
- x = np.arange(0, 2*(np.pi), 0.1)
- # setting the corresponding y - coordinates
- y = np.sin(x)
- # potting the points
- plt.plot(x, y)
- # function to show the plot
- plt.show()



np.arrange(start,stop,step):

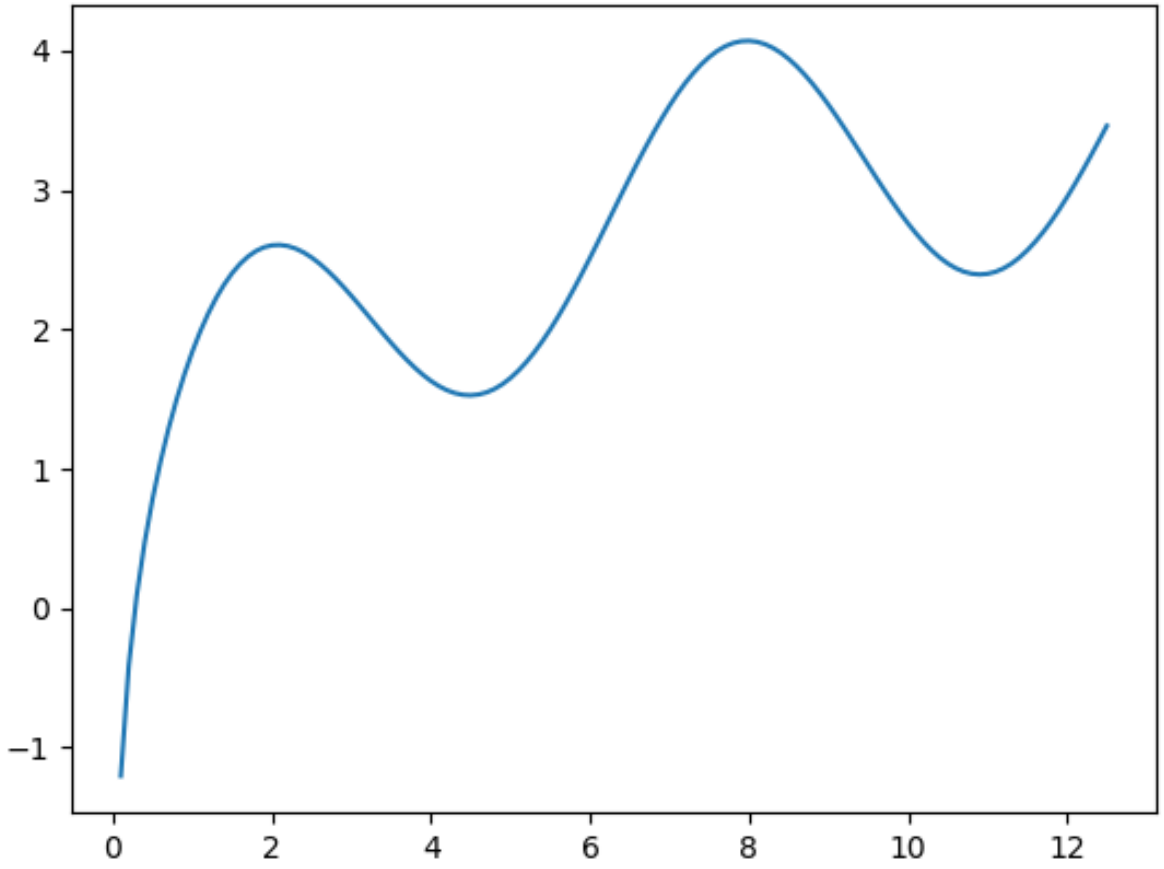
- np.arange(start, stop, step) is a NumPy function that returns evenly spaced values within a half-open interval [start, stop).
- start = 0: The array begins at 0.
- stop = 2*(np.pi): The array goes up to 2π (~6.283), but does not include it.
- step = 0.1: Each value increases by 0.1 from the previous one.

np.linspace(start,stop,num):

- start: The first value in the sequence.
- stop: The last value in the sequence (included).
- num: How many evenly spaced values to generate between start and stop.
- Key difference from np.arange : np.linspace generates exactly num points, including the end point. np.arange generates values with a fixed step size but may exclude the stop value depending on the step.

Plotting an equation:

- # importing the required modules
- import matplotlib.pyplot as plt
- import numpy as np
- # setting the x - coordinates
- x = np.arange(0, 4*(np.pi), 0.1)
- # setting the corresponding y - coordinates
- y = np.sin(x) + np.log(x) + 1
- # potting the points
- plt.plot(x, y)
- # function to show the plot
- plt.show()

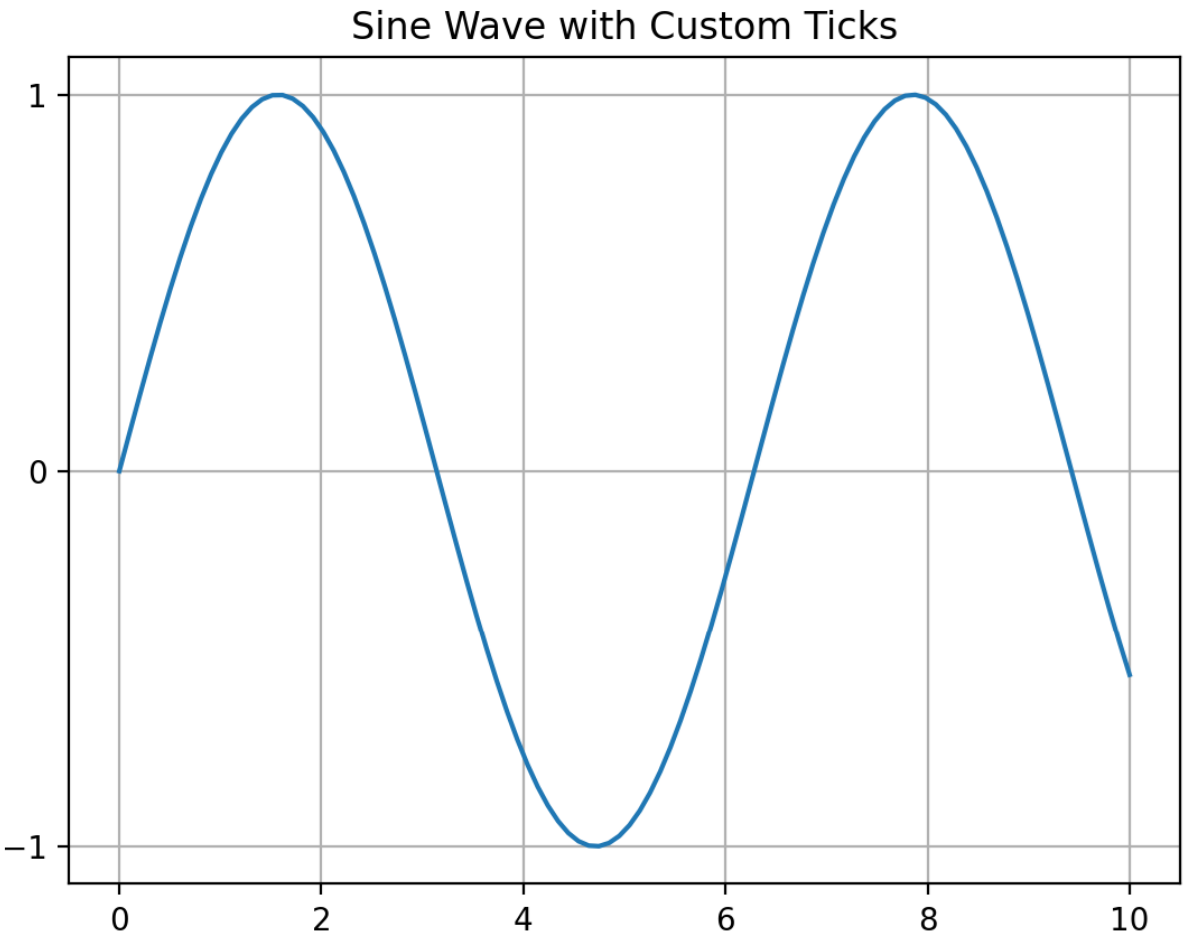


xtics and ytics:

- xticks() and yticks() are used to:
  - Set the positions of ticks on x or y-axis.
  - Label ticks with custom text.
  - Rotate or format tick labels.
  - Part of the matplotlib.pyplot module.
- Basic Syntax:
  - plt.xticks(ticks, labels=None, rotation=None)
  - plt.yticks(ticks, labels=None, rotation=None)
    - ticks: list of positions.
    - labels: list of strings for tick labels (optional).
    - rotation: angle to rotate tick labels (optional).

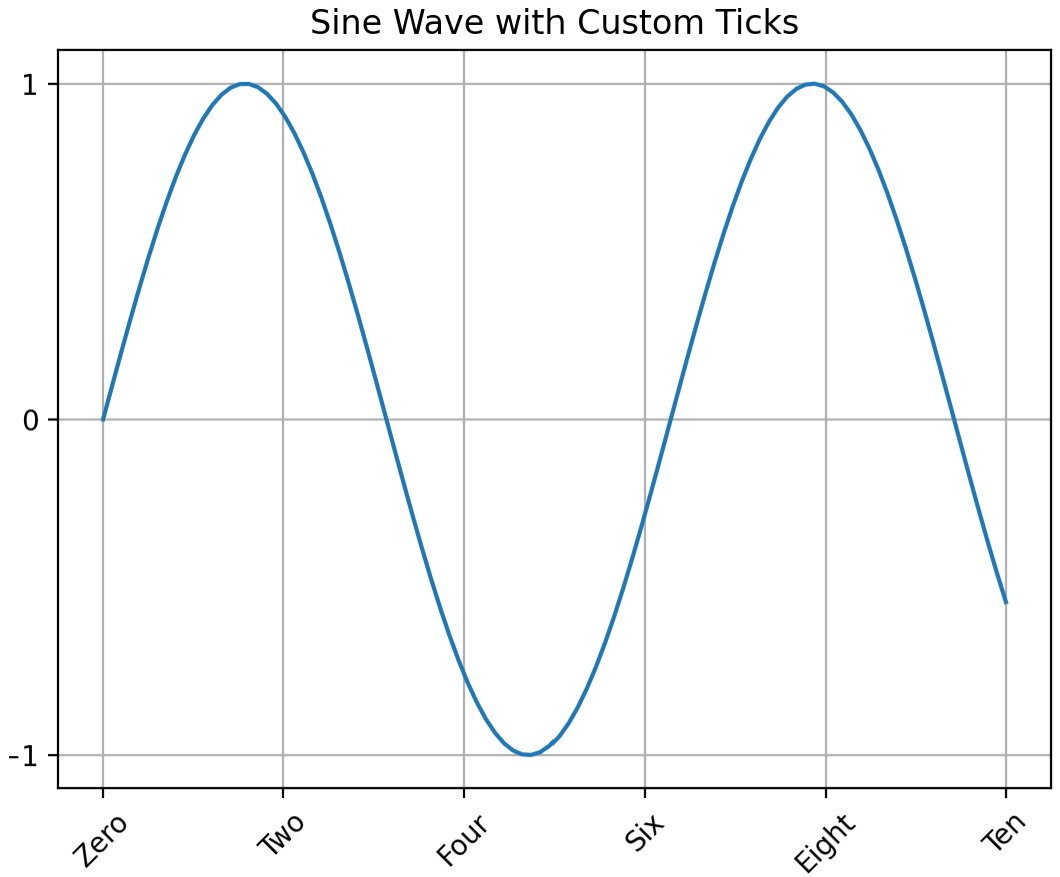
xtics and ytics example:

- import matplotlib.pyplot as plt
- import numpy as np
- x = np.linspace(0, 10, 100)
- y = np.sin(x)
- plt.plot(x, y)
- # Custom ticks on x and y axes
- plt.xticks([0, 2, 4, 6, 8, 10])
- plt.yticks([-1, 0, 1])
- plt.title("Sine Wave with Custom Ticks")
- plt.grid(True)
- plt.show()



xtics and ytics Example – Custome Labels and Rotation:

- import matplotlib.pyplot as plt
- import numpy as np
- x = np.linspace(0, 10, 100)
- y = np.sin(x)
- plt.plot(x, y)
- plt.xticks([0, 2, 4, 6, 8, 10],
- ['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten'],
- rotation=45)
- plt.yticks([-1, 0, 1])
- plt.title("Sine Wave with Custom Ticks")
- plt.grid(True)
- plt.show()



xtics and ytics Example – Custome Labels and Rotation:

- Instead of plt.xticks([0, 2, 4, 6, 8, 10], ['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten'], rotation=45) you can use:

```
x = [0,2,4,6,8,10]
Categories = ['Zero', 'Two', 'Four', 'Six', 'Eight', 'Ten']
plt.xtics(x)
ax.set_xticklabels(categories, rotation=-45)
```

Different Markers in Matplotlib:

Symbol	Marker	Description
'o'	Circle	Default point marker
'.'	Point	Small dot
','	Pixel	Very small dot
'x'	X	Cross
'+'	Plus	Plus sign
'*'	Star	Asterisk
's'	Square	Square marker
'D'	Diamond	Regular diamond
'd'	Thin diamond	Narrow diamond
'v'	Triangle down	Triangle pointing down
'^'	Triangle up	Triangle pointing up
'<'	Triangle left	Triangle pointing left
'>'	Triangle right	Triangle pointing right
'h'	Hexagon 1	
'H'	Hexagon 2	
'p'	Pentagon	

Color Codes in Matplotlib:

Code	Color	Description
'b'	Blue	
'g'	Green	
'r'	Red	
'c'	Cyan	Blue-green
'm'	Magenta	Pink-purple
'y'	Yellow	
'k'	Black	
'w'	White	(background)

Color Names in Matplotlib:

Color Name	Display
'blue'	
'green'	
'red'	
'cyan'	(blue-green)
'magenta'	
'yellow'	
'black'	
'white'	
'orange'	
'purple'	
'brown'	
'pink'	
'gray'	
'olive'	Greenish-yellow
'navy'	Dark blue

Color Names in Matplotlib:

Color Name	Display
'gold'	✨
'lime'	Bright green
'teal'	Blue-green
'coral'	Orange-pink
'indigo'	Dark violet

Hexadecimal Color Codes in Matplotlib:

Hex Code	Color
'#FF0000'	Red
'#00FF00'	Green
'#0000FF'	Blue
'#FFFF00'	Yellow
'#00FFFF'	Cyan
'#FF00FF'	Magenta
'#FFA500'	Orange
'#800080'	Purple
'#808080'	Gray
'#000000'	Black
'#FFFFFF'	White

Line Styles in Matplotlib:

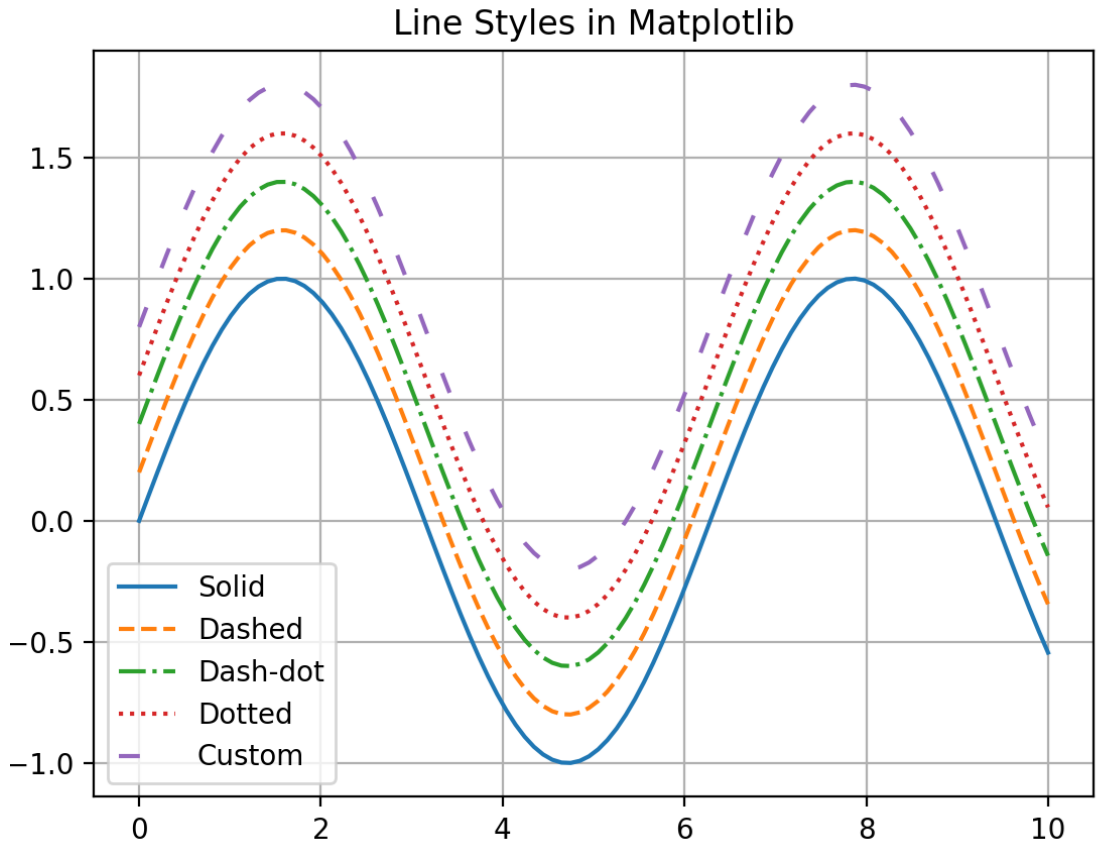
Line Style Code	Description	Example Look
'-'	Solid line	————————
'--'	Dashed line	— — — — —
'-.'	Dash-dot line	— · — · —
':'	Dotted line	·····
" or None	No line (markers only)	

Line Styles in Matplotlib:

- You can define custom dash patterns using a tuple:
  - (line_length, gap_length, ...)
  - plt.plot(x, y, linestyle=(0, (3, 5, 1, 5)))
    - This pattern means:
      - 3 units dash
      - 5 units space
      - 1 unit dash
      - 5 units space

Tuple	Style	Description
(0, (1, 1))	Dotted	Small dot style
(0, (5, 5))	Dashed	Even dash/space
(0, (3, 1, 1, 1))	Dash-dot-dot	Custom combo
(0, (4, 2, 1, 2))	Long dash + dot	Stylish variation

- import matplotlib.pyplot as plt
- import numpy as np
- x = np.linspace(0, 10, 100)
- y = np.sin(x)
- plt.plot(x, y, label='Solid', linestyle='-')
- plt.plot(x, y + 0.2, label='Dashed', linestyle='--')
- plt.plot(x, y + 0.4, label='Dash-dot', linestyle='-.')
- plt.plot(x, y + 0.6, label='Dotted', linestyle=':')
- plt.plot(x, y + 0.8, label='Custom', linestyle=(0, (5, 10)))
- plt.legend()
- plt.title("Line Styles in Matplotlib")
- plt.grid(True)
- plt.show()



Thank you!

Email: anand@iitbhilai.ac.in